

Message Volume Balancing in Repeated Sparse Collectives via Communication Task Sharing

Serdar Ozata^a, Kutay Tasci^{b,1}, Cevdet Aykanat^{a,*}

^aComputer Engineering Department, Bilkent University, Ankara, 06800, Turkey

^bDepartment of Computer and Data Sciences, Case Western Reserve University, Cleveland, 44106, OH, United States

Abstract

This work aims to increase the scalability of irregularly sparse bandwidth-bound applications, where the collective communication patterns induced by the task/data partition have sparse and irregular neighborhoods, resulting in high variation in processes' communication volume loads. Tier-0 HPC systems have low network diameters, helping to avoid message congestion. Thus, scaling such operations on HPC systems can be achieved by decreasing the computation and communication volume loads of bottleneck processes. We propose a two-phase framework: in the first phase, we employ an existing graph/hypergraph partitioning tool; in the second phase, our formulation introduces flexibility for owner processes to share their expand tasks with neighboring processes. We exploit this flexibility to reduce the communication load of bottleneck processes without increasing the total communication volume. In this expand-task sharing formulation, the processes' receive volumes remain unchanged, which restricts optimization to decreasing the maximum communication volume. To address this, we enhance our task-sharing method with a partial-reduce technique based on a vertex-cover model between each pair of communicating processes. Extensive evaluations with row-parallel SpMM, used as a sample application, on two large-scale HPC systems show that our framework significantly reduces the maximum communication volume, bringing it closer to the average across processes. For the SpMM instance uk-2002 on 4096 Lumi processes, our proposed method reduces the maximum communication volume by 86%, corresponding to a 61% decrease in parallel runtime. These results highlight the effectiveness of our communication optimization strategies in alleviating bottlenecks and improving scalability in bandwidth-bound applications.

Keywords: Sparse collectives, Communication volume balancing, Communication task sharing, Vertex cover, Distributed-memory parallel systems, Bandwidth-bound irregular applications, SpMM

1. Introduction

Sparse expand (scatter) and/or reduce types of communication operations arise in scientific computing and machine learning applications, such as sparse-matrix dense-matrix multiplication (SpMM), sparse matrix multiplication (SpGEMM), tensor decomposition, and graph neural network (GNN) training. As a typical example, we will consider the SpMM kernel of the form $Y=AX$ for clarity. Here, $A=(a_{ij})$ is an $n \times n$ square sparse matrix and X and Y are respectively dense input and output matrices of size $n \times f$, where $f \ll n$ making X a tall-and-skinny matrix. The focus of this work is to reduce the communication overhead of bandwidth-bound applications, and the SpMM kernel can be tuned to become bandwidth-bound by increasing column size (f) of X and Y .

SpMM is usually performed repeatedly/iteratively so that the input matrix X to be used for the next iteration is obtained from the current output matrix Y via linear matrix operations. To avoid communication during these linear matrix

operations, rows of X and Y are partitioned conformably. Furthermore, SpMM computations involving the same sparse matrix entail collective communication operations with identical sparsity patterns. Therefore, we preprocess the sparsity pattern once and reuse it across multiple SpMM computations.

We will briefly describe the 1D row- and column-parallel SpMM algorithms to clarify the expand or reduce operations involved. Row-parallel SpMM is based on partitioning the rows of matrix A and conformal partitioning of Y -matrix rows. Row-parallel SpMM involves a pre-communication phase followed by a computation phase. In the pre-communication phase, each process expands its local X -matrix rows to those processes that need them for their local SpMM calculations to compute their Y -matrix rows. The local SpMM performed by each process corresponds to the pre-multiplication of its local A -matrix rows with the dense X matrix. Column-parallel SpMM is based on partitioning columns of matrix A and conformal partitioning of X -matrix rows. Column-parallel SpMM involves a communication-free pre-computation phase followed by a post-communication phase. In the first phase, each process performs a local SpMM, which corresponds to multiplying its local A columns with its local X rows, to compute possibly partial results for Y . In the second phase, these partial results produced for Y rows are reduced so that each process computes the final

*Corresponding author

Email addresses: serdar.ozata@bilkent.edu.tr (Serdar Ozata), kutay.tasci@case.edu (Kutay Tasci), aykanat@cs.bilkent.edu.tr (Cevdet Aykanat)

¹This work was done when Kutay Tasci was with Bilkent University.

43 results for its local Y rows.

44 In irregularly sparse computations, the individual expand
45 and/or reduce communication operations induced by the task
46 and data partitioning among processes will lead to processes
47 performing collective communication operations with unstruc-
48 tured neighborhoods. Unstructured neighborhoods for collec-
49 tive communication operations incur high variation in the num-
50 ber and, more importantly, the volume of messages handled
51 (sent and received) by processes. For example, one process may
52 carry a disproportionately high volume load than the others.
53 Figure 1a shows such an example on a row-parallel SpMM in-
54 stance (sparse matrix pattern1 and $f=64$) on 112 processes,
55 where task/data partition is obtained by using the graph parti-
56 tioning tool MeTiS [1] with the objective of minimizing the to-
57 tal message volume while maintaining computational load bal-
58 ance. Sparse matrix pattern1 contains 19K rows/columns and
59 9M nonzeros with average and maximum row-degree values of
60 485 and 6.0K, respectively. The message volume load distri-
61 bution among processes displayed in Fig. 1a shows that process
62 92 is the bottleneck process with a message volume load of 26K
63 words, which is about 6 times larger than the average message
64 volume load of 4.7K words. In such a scenario, the bottleneck
65 process will have the potential to disrupt the scalability of the
66 overall parallelization.

67 Tier-0 HPC systems have low diameters, thus avoiding the
68 possibility of message congestion. Hence, scaling such opera-
69 tions on HPC systems can be achieved by reducing the compu-
70 tation and communication loads of the most heavily loaded
71 processes. Since the total computational cost is usually con-
72 stant, the former metric reduces to computational load balanc-
73 ing. However, total communication volume depends on parti-
74 tioning, so we formulate communication volume balancing to-
75 gether with total volume minimization. Although the existing
76 graph/ hypergraph partitioning tools successfully encode the
77 minimization of total communication volume while maintain-
78 ing computational load balance, they fail to address communi-
79 cation volume balancing.

80 We fill this gap in the literature by proposing a two-phase
81 framework for expand-task sharing. For the first phase, we use
82 an existing graph/hypergraph partitioning tool. The commu-
83 nication requirements of the task/data partition obtained here
84 are decoded into a list of sparse collective operations. For
85 the second phase, we propose a formulation based on owner
86 processes sharing their expand tasks with their neighbor pro-
87 cesses. We exploit this flexibility to minimize the volume load
88 of the most heavily loaded (bottleneck) processes. Restricting
89 expand-task sharing among collective-process neighborhoods
90 prevents an increase in the total communication volume be-
91 yond that achieved by the initial task/data partition. Further-
92 more, the proposed framework does not disturb the conformal
93 input/output partition obtained in the first phase.

94 Figure 1b displays the volume load distribution among 112
95 processes achieved by the proposed expand-task sharing algo-
96 rithm on pattern1. Comparison of Figs. 1a and 1b shows
97 how the proposed task-sharing algorithm balances processes'
98 volume loads, decreasing the maximum message volume load
99 from 26K words to 17K words without increasing the total mes-

sage volume. The proposed task-sharing algorithm decreases
the parallel runtime of this SpMM instance from 4.53 millisec-
onds to 3.56 milliseconds on MareNostrum 5.

Task decomposition in scale-free applications may produce
task-interaction graphs (TIGs) having large variations in the de-
grees of input vertices, with possibly a few input vertices with
very high degrees. Here, the degree of an input vertex refers
to the number of tasks that need the respective vertex. When a
high-degree input is assigned to a process p_k , its send volume
can be decreased through task sharing, but process p_k still re-
ceives a large amount of data proportional to the degree of this
input, which often becomes the communication bottleneck. To
alleviate this problem, other processes that hold the neighbors
(halos) of the high-degree vertices at p_k perform partial local
reduces before sending data, thereby decreasing the receive vol-
ume at process p_k . For this purpose, we utilize a vertex-cover
model on the bipartite graphs constructed between the boundary
vertices of each pair of communicating processes. A minimum
vertex cover with the smallest number of nodes on the receiving
process is chosen to achieve the least number of local reduces
while improving the receive volume at the same rate.

Figure 1c shows the message volume load distribution at-
tained by the vertex-cover model together with the expand-task
sharing scheme. Comparison of Figs. 1b and 1c shows how the
proposed vertex-cover model decreases processes' maximum
message load by decreasing the average volume load. The pro-
posed vertex-cover model, along with the task-sharing method,
further decreases the runtime of the aforementioned SpMM in-
stance from 3.56 to 1.75 milliseconds.

The performance of the communication optimization algo-
rithms is tested on row-parallel SpMM. Extensive experiments
are done on two tier-0 HPC systems, Lumi (up to 4096 cores)
and MareNostrum 5 (up to 7168 cores), using a wide range of
realistic datasets. The results of these experiments show that
the proposed communication scheme leads to significant im-
provement in parallel runtimes, thus increasing the scalability
of these applications. To support reproducibility and encour-
age further use, we will release our proposed communication
scheme as a library.

The rest of the paper is organized as follows. Table 1 sum-
marizes the notations used throughout the paper. The applica-
tion framework is presented in Section 2. Section 3 describes
the proposed communication optimization models and meth-
ods: expand-task sharing, vertex-cover-based partial reduce.
We briefly introduce reduce-task sharing in Section 3.5. Exper-
iments conducted to validate the performance of the proposed
methods are discussed in Section 4. Section 5 summarizes the
related work. Finally, the paper is concluded in Section 6.

2. Application Framework

The target application to be parallelized is considered as a
three-tuple $(\mathcal{T}, \mathcal{I}, \mathcal{O})$. Here, $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ denotes the set of
atomic tasks, $\mathcal{I} = \{i_1, i_2, \dots, i_n\}$ denotes the set of inputs to be
used by the tasks, and finally $\mathcal{O} = \{o_1, o_2, \dots, o_n\}$ denotes the set
of results to be computed by the tasks. The number of tasks,
inputs, and outputs is assumed to be all equal for simplicity.

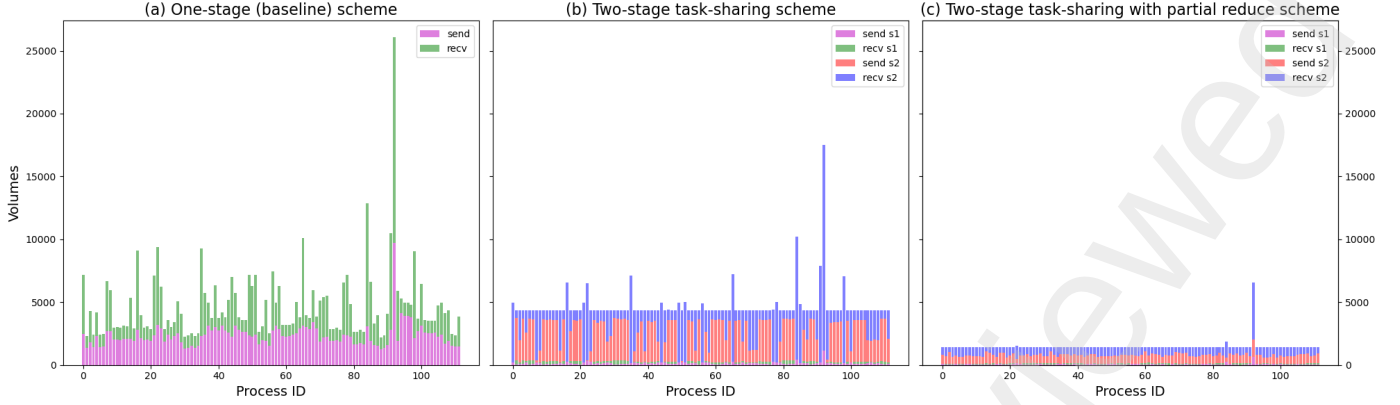


Figure 1: Volume load distribution incurred by (a) baseline, (b) two-stage task-sharing, (c) two-stage task-sharing with partial reduce for pattern1 on 112 processes

Table 1: Notations used throughout the paper

notation	explanation
K	number of processes
$\Pi(\mathcal{T})$	K -way task partition
$\Pi(I/O)$	Input/Output data partition
$input(t_j)$	set of inputs needed by task t_j
$output(t_j)$	set of intermediate/final results computed by t_j
$tasks(i_h)$	set of tasks that need i_h
$tasks(o_h)$	set of tasks that contribute to computation of o_h
$process(t_j)$	process to which task t_j assigned
MVC	minimum vertex cover
1s	conventional single stage expand comm. scheme
rb1s[2]	1s with apriori volume balance recursive bipartition
1spr	1s scheme with partial reduce
2s	proposed 2-stage expand comm. scheme
2spr	proposed 2s with partial reduce

In this formulation, $input(t_j)$ denotes the set of inputs needed by t_j and $output(t_j)$ denotes the set of outputs contributed by t_j . $tasks(i_h) = \{t_j : i_h \in input(t_j)\}$ denotes the set of tasks that need input i_h for execution, and $tasks(o_h) = \{t_j : o_h \in output(t_j)\}$ denotes the set of tasks that contribute to the computation of o_h .

For row-by-row-product-based task decomposition of SpMM [3]: t_j refers to the task of computing the inner product of A -matrix row j with the columns of dense input matrix X ; $input(t_j) = \{x_{k*} : a_{jk} \neq 0\}$, where x_{k*} denotes the k th row of dense matrix X ; $output(t_j) = \{y_{j*}\}$ is the final result computed by t_j . For outer-product-based task decomposition of SpMM [4]; t_j refers to the task of computing the outer product of A -matrix column j with dense input matrix row x_{j*} ; $input(t_j) = \{x_{j*}\}$, $output(t_k) = \{y_{k*}^j : a_{kj} \neq 0\}$. Here, $output(t_j)$ denotes the set of intermediate results computed by t_j for the Y -matrix rows corresponding to the row indices of the nonzeros at column j of A . Here, row-by-row parallel and outer-product parallel, respectively, correspond to the generalization of the row-parallel and column-parallel SpMV [5].

The target parallel system is a homogeneous distributed-memory system with a process set $\mathcal{P} = \{p_1, p_2, \dots, p_K\}$, where K processes communicate via message passing. The proposed

two-phase framework assumes obtaining K -way task partition $\Pi(\mathcal{T}) = \{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_K\}$ and conformal input/output data partition $\Pi(I/O) = \{I_1/O_1, I_2/O_2, \dots, I_K/O_K\}$ in the first phase. Here, Π denotes a K -way partition, where K parts are mutually disjoint and exhaustive.

Without loss of generality, $\Pi(\mathcal{T})$ and $\Pi(I_k/O_k)$ are decoded as assigning tasks in $\mathcal{T}_k$ and inputs/outputs in I_k/O_k to the same process p_k . So p_k owns the input data elements in I_k and is responsible for performing the tasks in $\mathcal{T}_k$. Let $process(t_j) = \{p_k : t_j \in \mathcal{T}_k\}$ and $process(i_h/o_h) = \{p_k : i_h/o_h \in I_k/O_k\}$ respectively denote the process to which t_j and i_h/o_h are assigned.

We assume that the task decomposition adopted incurs atomic tasks, where each atomic task either contributes to only one distinct output element or needs only one distinct input data item. If needed, we can easily change the indices of the input and output elements such that $output(t_j) = \{o_j\}$ and $input(t_j) = \{i_j\}$ in the former and latter cases respectively. Thus, a proper task/data partition assigns o_h and task t_h to the same process in the former case, and it assigns i_h and task t_h to the same process in the latter case. The partitioning adopted in the former and latter cases can also be considered as output and input data partitioning with the owner-computes rule.

Figure 2 shows sample TIGs for the former and latter cases. Although TIGs are usually undirected, we draw them as three-stage directed graphs in order to clarify the input-to-task and task-to-output interactions. In Fig. 2a, $input(t_2) = \{i_1, i_2, i_3\}$ means t_2 needs i_1 , i_2 , and i_3 for execution. Each task t_i directly computes output o_i . In Fig 2b, each task t_j needs only

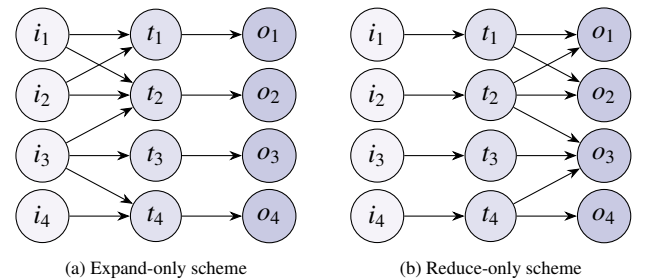


Figure 2: Task from/to input/output data interaction graphs

i_j . $output(t_4) = \{o_3, o_4\}$ refers to the fact that t_4 computes two possibly different intermediate results for o_3 and o_4 .

The former case incurs only expand-type communications on the shared input elements in a pre-communication phase. Here, an input element is said to be shared if it is required by more than one atomic task for execution. The latter case only incurs reduce-type communication operations based on the intermediate results generated by different processes for the shared output elements during the post-communication step. The former and latter cases, respectively, refer to the row-parallel and column-parallel SpMMs. In this work, we restrict our attention to the former case, which involves only expand-type communications. We briefly discuss the extension to the latter case, which involves only reduce-type communication, in Section 3.5. For clarity in the following discussion, Fig. 2a can be simplified by merging t_j and o_j into a single vertex, denoted t_j/o_j or simply o_j .

The task and input/output partitions $\Pi(\mathcal{T})$ and $\Pi(\mathcal{I}/\mathcal{O})$ obtained in the first phase are decoded as a set of expand collectives as follows. For each i_h we compute:

$$needList(i_h) = \bigcup_{t_j \in task(i_h)} process(t_j). \quad (1)$$

If $needList(i_h) = process(i_h)$ then all tasks that need input i_h are in the same process with i_h so that i_h will not incur any communication. Otherwise, the owner process of i_h , $process(i_h)$, should expand input i_h to each process in $needList(i_h)$ except itself. Then the expand task e_h is added to the expand collective list as $ELIST = ELIST \cup \{e_h\}$, where

$$owner(e_h) = process(i_h) \quad (2)$$

$$pList(e_h) = needList(i_h) - owner(e_h) \quad (3)$$

and the expand tasks of process p_k can be denoted as,

$$elist(p_k) = \{e_h \in ELIST : owner(e_h) = p_k\}.$$

After identifying all expand collectives, we can compute the message destinations to be sent and received by a process p_k ,

$$SendList(p_k) = \{p_\ell \in pList(e_h) : p_k = owner(e_h)\} \quad (4)$$

$$RecvList(p_k) = \{p_\ell : p_k \in pList(e_h), p_\ell = owner(e_h)\}. \quad (5)$$

3. Proposed Communication Optimization Methods

3.1. Communication Costs Formulation

A well-defined cost model for communication is essential to ensure that the predicted benefits of combinatorial optimization algorithms align with actual performance in real applications. However, modern distributed HPC architectures feature highly complex hardware designs and functionalities, making the accurate modeling of communication costs extremely challenging. As a result, developing reliable cost models that generalize across different architectures is a fundamental requirement for effective optimization.

In bandwidth-bound problems within message-passing environments, two key metrics can be used to define the communication load of a process p_k : $SendVol(p_k)$ and $RecvVol(p_k)$. These metrics serve as the foundation for our communication cost model. The expressions for the send and receive volume loads of a process p_k can be derived from Eq. 3 as follows:

$$SendVol(p_k) = \sum_{e_h: owner(e_h)=p_k} size(i_h) \times |pList(e_h)| \quad (6)$$

$$RecvVol(p_k) = \sum_{i_h: p_k \in pList(e_h)} size(i_h). \quad (7)$$

In this setting, we consider three distinct cost models for modeling the communication load of each process:

$$VolLoad1(p_k) = SendVol(p_k) \quad (8)$$

$$VolLoad2(p_k) = \max\{SendVol(p_k), RecvVol(p_k)\} \quad (9)$$

$$VolLoad3(p_k) = SendVol(p_k) + RecvVol(p_k). \quad (10)$$

$VolLoad1$ (Eq. 8) defines the communication cost using only the send volume, making it particularly useful for implementations where the receive volume can be hidden through communication-computation overlap and/or when shared-memory protocols in message-passing interfaces (MPI) are leveraged for intra-node communication. $VolLoad2$ and $VolLoad3$ consider both $SendVol(p_k)$ and $RecvVol(p_k)$ as contributing equally to the communication cost, as expressed in Eq. 9 and Eq. 10. The difference is that $VolLoad2$ assumes that incoming and outgoing messages totally overlap in the local architecture of the individual process, so that the volume load is determined by the maximum of the incoming and outgoing message volumes. However, $VolLoad3$ assumes that incoming and outgoing messages do not overlap at all in an individual process because of the congestion in the local architecture components (bus, DRAM, etc.), so that the volume is determined by the sum of these two message volumes.

Each cost model is based on different assumptions regarding the underlying architecture and the implementation characteristics of the target tasks. In our experiments, $VolLoad3$ (Eq. 10) shows better corroboration between the proposed communication optimization methods and real communication runtimes. The optimization methods can easily be extended to the remaining and possibly other communication cost models.

3.2. Expand-Task Sharing

The proposed communication optimization is based on introducing flexibility to processes by sharing each of their expand tasks with a neighbor process in the expand list of that expand task. In particular, consider an expand task e_h assigned to process p_k , that is $owner(e_h) = p_k$. In the conventional scheme, process p_k will send i_h to each process in $pList(e_h)$, thus incurring communication volume load of $\lambda(e_h) \times size(i_h)$ to p_k . Here, $\lambda(e_h) = |pList(e_h)|$. In the proposed optimization, p_k sends i_h to a selected process $p_\ell \in pList(e_h)$ to share the responsibility of expanding e_h between p_k and p_ℓ based on their respective communication volume loads. Since p_k already holds i_h and p_ℓ

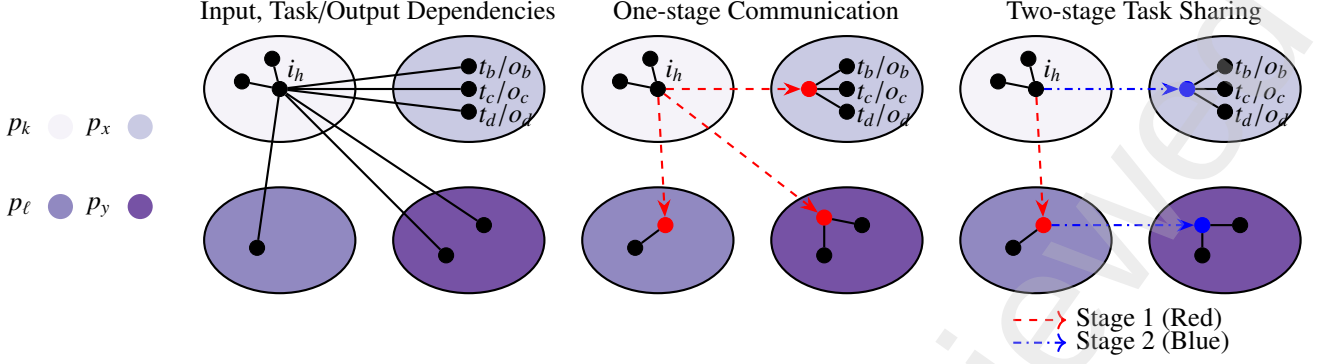


Figure 3: Comparison of the conventional one-stage (1s) and the proposed two-stage expand-task-sharing (2s) methods. In 1s, p_k expands i_h to $\{p_l, p_x, p_y\}$ in a single stage. In 2s, p_k shares expand task e_h with process p_l in the first stage, then they complete the remaining of the task in the second stage.

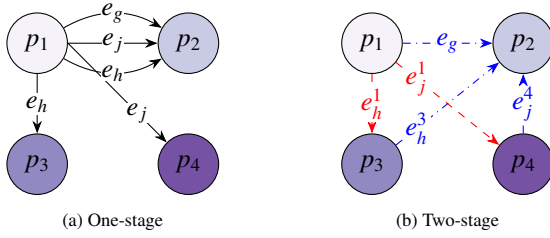


Figure 4: Communication of expand tasks e_g , e_h , and e_j . (a) Conventional one-stage method. (b) The proposed two-stage method: e_h^1 , e_j^1 are performed in the first stage and the remaining tasks in the second.

Algorithm 1 Iterative Expand Task Sharing

```

1: function TASKSHARING( $ELIST$ )
2:   for  $e_h \in ELIST$  in random order do
3:      $p_k \leftarrow \text{owner}(e_h)$ 
4:     Remove  $e_h$  from  $elist(p_k)$ 
5:      $VolLoad(p_k) \leftarrow VolLoad(p_k) - |pList(e_h)|$ 
6:     ASSIGN( $e_h$ )
7:   end for
8: end function

```

receives it from p_k , the two processes share the remaining communication of e_h : $pList(e_h) - \{p_l\}$. This reassignment decreases the send-volume load of p_k by up to $(\lambda(e_h) - 1) \times size(i_h)$ while increasing the send-volume load of p_l by the same amount. However, if this reassignment would cause p_l 's communication volume to exceed that of p_k , the task is shared such that both processes maintain the same communication volume. It is worth noting that restricting the reassignment of expand tasks to processes within $pList(e_h)$ ensures that the total communication volume induced by partition Π remains unchanged.

We provide Fig. 3 to exemplify the task sharing of e_h , where $owner(e_h) = p_k$ and $pList(e_h) = \{p_l, p_x, p_y\}$. Process p_k shares expand task e_h with p_l in such a way that process p_k sends i_h to p_k during the first communication stage. Then, p_k and p_l concurrently send i_h to p_x and p_y in the second stage.

Receive volume loads of processes do not change during these assignments for expand-task sharing. Hence, the above-mentioned amount of decreases/increases in the send-volume loads between a pair of processes directly equals the amount of increase and decrease in their total volume loads (according to Eq. 10). We provide Fig. 4 to show this fact. p_1 shares e_h ($pList(e_h) = \{p_2, p_3\}$) and e_j ($pList(e_j) = \{p_2, p_4\}$) with p_3 and p_4 , respectively. As seen in Fig. 4b, the receive volume loads of sharing processes p_3 and p_4 do not change since they receive i_h and i_j in the first stage of the communication.

Algorithms 1 and 2 show the expand-task sharing algorithm. Algorithm 1 calls our ASSIGN procedure for each expand task in random order. With the resulting task assignment, the processes are not necessarily responsible for the expand tasks given in the initial task/data partition. Hence, the original $ELIST$ nota-

tion defined in Eq. 4 is altered, as completing each expand task e_h might be divided between two processes p_k and p_l where the respective partial expand tasks are e_h^k and e_h^l . The union of these pairs is the original expand task e_h .

In Algorithm 2, for each expand task $e_h \in ELIST$ where $owner(e_h) = p_k$, our ASSIGN algorithm selects the process $p_l \in pList(e)$ that has the minimum volume load in line 3. The objective of this heuristic is to balance the communication load of a process p_k by sharing its expand task e_h . For this purpose, line 4 computes the volume load difference (Δ) between the owner and the sharing process. Then it compares Δ against connectivity $\lambda(e_h)$ of the expand task in the if-then-else block between lines 6-14. If $-\Delta$ is so large that $VolLoad(p_l)$ still becomes smaller than $VolLoad(p_k)$ after the assignment (line 8), the owner assigns all the load of expand task e_h to the sharing process. Otherwise, the load is divided between the owner and the sharing process, so that $|VolLoad(p_k) - VolLoad(p_l)| \leq 1$ (line 10). Lastly, if the owner process is the one with the minimum volume (including the expand task e_h), the expand task is not shared (line 13). In lines 15-17, $\Lambda_{\gamma-1}(e_h)$ refers to the first $\gamma - 1$ processes in $pList(e_h)$ excluding p_l . Finally, lines 18-21 perform the necessary updates in $pList$ and $VolLoad$.

In some cases, after applying the proposed expand-task sharing method, the bottleneck processes share all of their (shareable) expand tasks. Hence, these processes have only send volume in the first stage and receive volume in the second stage. For example, for the SpMM instance in Fig. 1b, it can be seen that the processes with large communication volumes, such as process 92, share all of their expand tasks. The least loaded processes then receive the tasks that they will share in the first stage

Algorithm 2 Expand Task Assignment

```

1: function ASSIGN( $e_h, EList$ )
2:    $p_k \leftarrow \text{owner}(e_h)$ 
3:    $p_\ell \leftarrow \arg \min_{p_x \in pList(e_h)} VolLoad(p_x)$ 
4:    $\Delta \leftarrow VolLoad(p_\ell) - VolLoad(p_k)$ 
5:    $\lambda(e_h) \leftarrow |pList(e_h)|$ 
6:   if  $\Delta < \lambda(e_h)$  or  $\lambda(e_h) > 1$  then
7:     if  $\Delta \leq -\lambda(e_h)$  then
8:        $\gamma \leftarrow 1$  ▷ Share all
9:     else
10:       $\gamma \leftarrow \lceil (\Delta + \lambda(e_h)) / 2 \rceil$  ▷ Share w.r.t.  $\Delta$ 
11:    end if
12:  else
13:     $\gamma \leftarrow \lambda(e_h)$  ▷ Do not share
14:  end if
15:   $\Lambda(e_h) \leftarrow pList(e_h) - \{p_\ell\}$ 
16:   $pList(e_h^k) \leftarrow \Lambda_{\gamma-1}(e_h) \cup \{p_\ell\}$ 
17:   $pList(e_h^\ell) \leftarrow \Lambda(e_h) - \Lambda_{\gamma-1}(e_h)$ 
18:   $EList(p_k) \leftarrow EList(p_k) \cup \{e_h^k\}$ 
19:   $EList(p_\ell) \leftarrow EList(p_\ell) \cup \{e_h^\ell\}$ 
20:   $VolLoad(p_k) \leftarrow VolLoad(p_k) + \gamma \times size(i_h)$ 
21:   $VolLoad(p_\ell) \leftarrow VolLoad(p_\ell) + (\lambda(e_h) - \gamma) \times size(i_h)$ 
22: end function

```

(recv s1 in Fig. 1 legend) and perform these tasks in the second stage along with their own expand tasks (send s2 in Fig. 1 legend). Even though process 92 was able to share all of its expand tasks, its volume load is well over the average. This problem is addressed in the next section.

3.3. Partial Reduce

Although the proposed expand-task sharing method decreases the send volume of the bottleneck process(es), the receive volumes of the processes remain unchanged, which restricts the overall effort towards decreasing the maximum communication volume. Furthermore, task decomposition in scale-free applications may produce large variations in the degrees of input vertices, with possibly a few high-degree input vertices in the TIG. Here, the degree of an input vertex refers to the number of tasks that need the respective vertex. In row-parallel SpMM, this corresponds to a sparse matrix with a number of almost dense rows/columns. When a high-degree input i_h is assigned to a process p_ℓ , the send volume incurred by expand task e_h is proportional to its process connectivity $\lambda(e_h)$. However, the receive volume incurred by t_h/o_h is proportional to the degree of o_h which might be much larger than $\lambda(e_h)$. Although its send volume can be decreased through task sharing, p_ℓ still receives a large amount of data proportional to the degree of the input, which often becomes a communication bottleneck. To alleviate this problem, other processes that hold some of the inputs in $tasks(o_h)$ perform partial local reduces before sending data, thereby decreasing the receive volume at p_ℓ .

3.3.1. Heterogeneous Messages

For each message from p_k to p_ℓ , the objective of the method is to minimize the volume of this message, thus decreasing the

send volume of p_k and decreasing the receive volume of p_ℓ by moving the computation tasks associated with the output items $t_h/o_h \in \mathcal{T}_k/O_k$ to the owner of the input items that are needed for producing o_h . The owners then send the partial computation results (reduces) to the receiver. In this way, the communication scheme produces heterogeneous messages that involve input values as well as partial results. Note that expand-only and reduce-only schemes produce homogeneous messages that involve either input values or partial results.

The purpose of partial reduces is to remove expand tasks that are needed by p_ℓ to decrease the receive-volume load of p_ℓ , which is exactly equal to the decrease in the send-volume load of p_k . However, process p_ℓ can be removed from $pList(e_h)$ if no task in p_ℓ remains that need i_h for producing its respective output. In other words,

$$i_h \notin \bigcup_{t_j: p_\ell \in process(t_j)} tasks(t_j). \quad (11)$$

Each reduce operation incurs an extra volume load for the sender and receive processes. Hence, for each communicating process pair, an intelligent algorithm is needed to find an optimal set of input-item subsets for which partial results are to be computed and sent to the receiving process. We introduce a vertex-cover-based partial reduce method that allows us to find such an optimal set. Here, optimality refers to a reduce selection scheme that minimizes the communication volume from p_k to p_ℓ .

3.3.2. Minimum Vertex Cover (MVC)

For each communicating process pair p_k and p_ℓ , two bipartite graphs $G_{k\ell}$ and $G_{\ell k}$ are constructed, where in some corner cases, one of these two bipartite graphs can be empty. The bipartite graph $G_{k\ell}$, which represents the content of the message from p_k to p_ℓ , is defined as follows:

$$G_{k\ell} = (V_{k\ell} = A_{k\ell} \cup B_{k\ell}, E_{k\ell}), \quad (12)$$

where $A_{k\ell}$ and $B_{k\ell}$ constitute the two parts of the vertex set:

$$A_{k\ell} = \{i_h \in \mathcal{I}_k : i_h \in \bigcup_{t_j \in \mathcal{T}_\ell} input(t_j)\} \quad (13)$$

$$B_{k\ell} = \{o_h \in \mathcal{O}_\ell : t_h \in \bigcup_{i_j \in \mathcal{I}_k} tasks(i_j)\}. \quad (14)$$

Here, $A_{k\ell}$ denotes the boundary input vertices of p_k with respect to p_ℓ , where a vertex is said to be boundary if it is needed by at least one task assigned to another process. In a dual manner, $B_{k\ell}$ denotes the boundary output vertices of p_ℓ , with respect to p_k . Finally, the edge set $E_{k\ell}$ can be given as,

$$E_{k\ell} = \{(i_j, o_h) \in A_{k\ell} \times B_{k\ell} : t_h \in tasks(i_j)\}. \quad (15)$$

The proposed partial reduce method generates a minimum vertex cover for each nonempty subgraph $G_{k\ell}$.

Each edge in the bipartite graph $G_{k\ell}$ represents the input-output dependency of the p_k -to- p_ℓ message. Since vertices in a

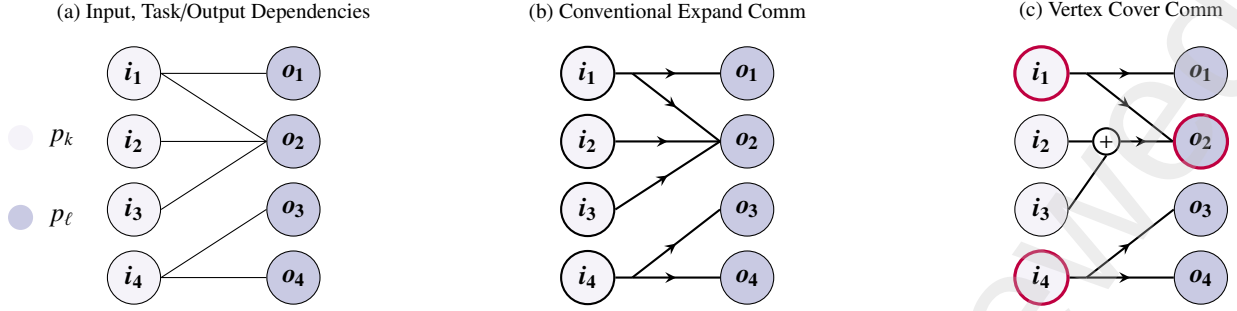


Figure 5: Partial reduce in expand-task sharing. (a) Bipartite dependency graph from p_k to p_l . (b) Conventional method: p_k expands $\{i_1, i_2, i_3, i_4\}$ to p_l . (c) Proposed partial reduce method for decreasing the receive volume of p_l , minimum vertex cover (in purple) $MVC = \{i_1, o_2, i_4\}$, process p_k sends $\{i_1, i_4\}$ and reduce partial result for i_2/o_2 to p_l . This decreases the receive volume of p_l from 4 to 3.

vertex cover are incident to all edges, any vertex cover satisfies the communication requirement. An MVC will then minimize the communication volume of the message. Among all possible MVCs, we are looking for the one that minimizes the number of reduce operations in order to minimize the imbalance due to carrying some computations from p_l to p_k . Such an MVC also provides more flexibility to the expand-task sharing algorithm compared to other MVCs of G_{kl} . Below, we discuss what type of vertex cover satisfies this optimization objective by showing how a given vertex cover is decoded to determine the contents of the message from p_k to p_l .

Let C be an MVC of G_{kl} and $C_A = C \cap A_{kl}$, $C_B = C \cap B_{kl}$. In the partial reduce method, p_k sends each vertex $i_h \in C_A$ to p_l to satisfy the edges incident to i_h . Then, all remaining edges are satisfied via reduce operation on the vertices in C_B . Figure 5c shows a sample bipartite graph with an MVC: $C_A = \{i_1, i_4\}$ and $C_B = \{o_2\}$. C_A is decoded as p_k sending/expanding i_1, i_4 to p_l thus satisfying all edges except (i_2, o_2) and (i_3, o_2) . These remaining two edges incident to $o_2 \in C_B$ are satisfied via p_k performing a local reduce operation on i_2, i_3 and then sending the partial result to p_l . The comparison of Fig. 5b and Fig. 5c shows that the proposed vertex-cover method decreases the volume of the p_k -to- p_l message from 4 to 3. The above-mentioned communication method clearly shows that minimizing the number of local reduce operations on the sending process can be achieved by finding the MVC with the maximum number of vertices on the sending side (C_A) of the bipartite graph, since the number of partial results that will be sent from p_k to p_l is equal to $|C_B|$. The MVC algorithm given in Algorithm 3 finds such an MVC. We provide the proof for this in Appendix A.

In Algorithm 3, the Hopcroft-Karp algorithm [6] is used to

find a maximum cardinality matching. Hopcroft-Karp algorithm runs in $O(\sqrt{VE})$ time for each bipartite graph $G = (V, E)$. This determines the running time of Algorithm 3.

3.4. Combining Expand-Task Sharing and Partial Reduce

The proposed vertex-cover-based partial reduce method decreases message sizes with a minimal amount of disturbance to the computational load balance determined by the initial task/data partition obtained in the first phase. However, it does not maintain the overall balance of the communication volume. Therefore, it is implemented as a pre-processing step to the proposed task-sharing method. We should report here that we have also experimented with applying first the task-sharing method and then applying the partial-reduce method. This alternative scheme produced worse performance in terms of communication load metrics and parallel runtime. This experimental result was expected as the vertex-cover-based partial reduce method performed in the second phase disturbs the communication volume balance attained in the first phase.

Consider a partial reduce result o_h^k computed by p_k according to the MVC found in G_{kl} . As an example, in Fig. 5c, $o_2^k = i_2 \oplus i_3$ is a partial reduce result computed by p_k for output o_2 on behalf of p_l . Consider another partial result $o_j^k = i_2 \oplus i_3$ to be computed for $o_j \in O_x$ from the MVC found in G_{kx} . Although o_2^k and o_j^k require the same set of input items, their aggregations might be unique to their respective task/output items depending on the application. For example, SpMM with different nonzero values in the sparse matrix and GNN with edge-weighted graphs constitute such applications. In fact, this corresponds to task input/output data interaction graphs shown in Fig. 2 having weighted edges. Therefore, these partial results are not subject to the task-sharing method that comes after.

For edge unweighted (boolean) task input/output data interaction graphs, the reduce operations that require the same set of input items produce the same partial result. Each such set of partial reduce results needs to be computed only once, can be considered as a single expand task, thus can be utilized by the task-sharing method. Although this optimization is viable, we found that a very small number of reduce tasks match. Therefore, it does not impact the overall performance. The main problem is that, for a given process p_k , Algorithm 3 finds an

Algorithm 3 Minimum Vertex Cover

```

1: function MINIMUMVERTEXCOVER( $G = (V = A \cup B, E)$ )
2:    $M \leftarrow \text{MAXIMUMCARDINALITYMATCHING}(G)$ 
3:    $F_\ell \leftarrow \{v \in A : v \text{ is unmatched in } M\}$ 
4:    $U \leftarrow \{v \in V : \text{there exists an } M\text{-alternating path from some } v_\ell \in F_\ell \text{ to } v\}$ 
5:    $C \leftarrow (A - U) \cup (U \cap B)$ 
6:   return  $C$ 
7: end function

```

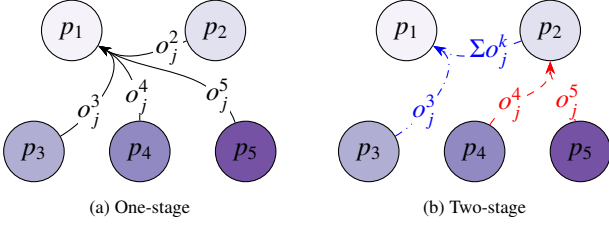


Figure 6: Reduce task-sharing. (a) One-stage method. (b) Proposed two-stage reduce-task-sharing method: o_j^4 and o_j^5 are sent to p_2 in stage 1, aggregated with o_j^2 (Σo_j^k), and sent to p_1 in stage 2. o_j^3 is not shared with p_2 and directly sent in stage 2.

MVC for each communicating process p_ℓ independently, without considering the other MVCs of p_k and their resulting partial reduces. We leave developing an algorithm for finding MVCs with the maximum amount of matching partial reduces as future work.

By incorporating the partial reduce method mentioned in Section 3.3, the proposed two-stage communication scheme consists of P2P messages that involve two communication stages interleaved with one computation substage.

- **Communication Stage 1:** the owner of the shared input items sends the respective input items to the sharing processes.
- **Computation Stage 1:** computes the partial reduce results that will be sent in the second stage.
- **Communication Stage 2:** consists of completing the remaining shared expand tasks and the non-shared partial reduce and expand tasks.
- **Computation Stage 2:** consists of processes performing the final aggregation on the input and partial output results.

3.5. Reduce-Task Sharing

So far, our optimization methods have only been applied to expand-only communication schemes. However, the proposed methods can also be adapted to a reduce-only scheme in Fig. 2b.

The vertex-cover method is easily adaptable, since it relies only on the bipartite dependency graphs between pairs of communicating processes to construct a new communication scheme. Consequently, it remains agnostic to the initially employed parallelization strategy.

To devise reduce-task sharing, we will introduce a dual variant of the expand-task sharing scheme. In this scheme, a partial output computed for o_i by p_k is shared with another process p_ℓ that also holds a partial output for o_i . Process p_ℓ then aggregates these partial outputs and sends the result to $process(o_i)$.

This duality between expand-task sharing and reduce-task sharing yields the following properties. As mentioned in Section 3.2, in expand-task sharing, the send volumes are balanced, whereas the receive volumes remain unchanged. In reduce-task sharing, the receive volumes are balanced, whereas the send volumes remain unchanged. Figure 6 demonstrates this. In two-stage reduce-task sharing, the receive volume of p_1 is decreased, whereas the send volumes of all processes remain unchanged.

Beyond these single-stage settings, many distributed SpMM implementations also employ 2D communication schemes,

where an expand-only stage is followed by a reduce-only stage. In such cases, our methods can be adapted to balance the volume loads in both stages and even merge them by leveraging the partial reduce method. A detailed investigation of the methods for 2D and column-parallel SpMM is left for future work.

4. Experiments

4.1. Example Application

We evaluate and validate the performance of the communication optimization algorithms proposed in Section 3 on SpMM operation. SpMM is a kernel operation widely used in scientific computing and graph analytics, and has recently gained popularity in machine learning applications. In SpMM, the dense matrix is tall and skinny. Besides the sparsity pattern of the sparse matrix, the column dimension of the dense matrix is also detrimental to the communication characteristics of parallel SpMM. The SpMM kernel becomes a bandwidth-bound application by increasing the column size of the input and output dense matrices.

We use the 1D row-parallel SpMM implementation for which the details of the communication requirements are given earlier. For the conventional one-stage communication scheme and the first phase of the proposed two-stage communication scheme, we use the multilevel graph partitioner MeTiS [1]. Here, the (task interaction) graph given to MeTiS is the graph whose adjacency matrix representation corresponds to the respective sparse matrix. The weight of each vertex is set equal to the number of nonzeros in the respective row of the sparse matrix. In this way, the partitioning constraint corresponds to computational load balancing, whereas the partitioning objective corresponds to minimizing the edge cut, which relates to decreasing the total communication volume. For the largest SpMM instance *arabic-2005*, we used ParHIP for partitioning as ParHIP is expected to perform better in large graphs [7, 8].

4.2. Experimental Setup

We implemented the SpMM application in C, utilizing the Message Passing Interface (MPI) for inter-process communication to harness parallel processing capabilities. To enhance computational performance, we integrated the Basic Linear Algebra Subprograms (BLAS) library, which offers optimized routines for fundamental linear algebra operations. This integration significantly accelerates processes' local computations.

To achieve good performance with non-blocking MPI collectives, we implemented several key strategies during MPI initialization. Recognizing that our approach increases the message count during communication, we enabled the “leave pinned” option in MPI. This setting helps mitigate the overhead associated with frequent memory registrations and deregistrations by keeping memory regions pinned, thereby reducing the latency for repeated communications [9]. Additionally, we addressed potential inefficiencies arising from bandwidth saturation due to congestion at socket boundaries. To minimize these inefficiencies, we employed process mapping and binding techniques

to assign processes to specific physical cores during initialization. This practice, known as processor affinity or CPU pinning, also ensures that each process consistently runs on a designated CPU, enhancing cache utilization and reducing context-switching overhead. By integrating these optimizations, leveraging the “leave pinned” option, and implementing precise process-to-core mappings, we aimed to enhance the efficiency of non-blocking MPI collectives, thereby improving the overall performance of our parallel applications.

For benchmarking, two state-of-the-art pre-exascale supercomputers were employed: LUMI and MareNostrum 5. In LUMI, we employed the LUMI-C partition, which consists of 1,536 nodes—each equipped with dual 64-core AMD EPYC 7763 processors (providing 128 cores per node) and configured with 256 GB of DDR4 memory (with select nodes offering up to 1 TB)—all interconnected via a high-performance Cray Sling-shot network. We utilized Cray’s MPICH implementation for MPI communications and CrayBLAS to accelerate local computations, both integrated within the Cray Programming Environment on Lumi. In MareNostrum 5, our experiments utilized the General-Purpose Partition (GPP), a CPU-only configuration comprising 6,408 nodes; each node is powered by dual Intel Sapphire Rapids 8480+ processors running at 2 GHz with 56 cores per processor (totaling 112 cores per node), paired with 256 GB of DDR5 memory (with a subset featuring up to 1 TB for high-memory tasks) and fast NVMe local storage. We utilized Intel MPI for MPI communications and Intel’s MKL for local computations. Together, these systems provide robust and scalable platforms for assessing the performance of traditional HPC workloads.

Expand-task sharing and vertex-cover-based partial-reduce methods are implemented in C++. OpenMP is utilized for shared-memory parallelization of finding the MVC of each pair of communicating processes. The communication procedure is generated once for each sparse matrix and can be used repeatedly by the SpMM application as needed.

Communication Implementation

For the conventional implementation of expand-type parallel communication, a single communication stage is employed to exchange data among processes. In this setting, each process first posts non-blocking receive calls (via `MPI_Irecv`) for all processes in its `RecvList`, then issues non-blocking send

Algorithm 4 One-Stage Expand Communication (1s)

```

1: function ONESTAGE(SendList, RecvList)
2:   for each  $p_\ell \in \text{RecvList}$  do
3:     MPI_Irecv(recvBuffer $_\ell$ ,  $p_\ell$ )
4:   end for
5:   for each  $p_\ell \in \text{SendList}$  do
6:     Fill sendBuffer $_\ell$ 
7:     MPI_Isend(sendBuffer $_\ell$ ,  $p_\ell$ )
8:   end for
9:   MPI_WaitAll on all receives
10: end function

```

Algorithm 5 Proposed Two-Stage Communication (2spr)

```

1: function TWOSTAGE(SendList1, SendList2, RecvList1, RecvList2)
2:   for each  $p_\ell \in \text{RecvList}_1$  do
3:     MPI_Irecv(recvBuffer $_{1,\ell}$ ,  $p_\ell$ )
4:   end for
5:   for each  $p_\ell \in \text{SendList}_1$  do
6:     Fill sendBuffer $_{1,\ell}$ 
7:     MPI_Isend(sendBuffer $_{1,\ell}$ ,  $p_\ell$ )
8:   end for
9:   if Partial Reduce then
10:    Perform Computation Stage 1
11:    MPI_WaitAll on all receives
12:    ▶ Expand tasks to be shared are all received
13:   end if
14:   for each  $p_\ell \in \text{RecvList}_2$  do
15:     MPI_Irecv(recvBuffer $_{2,\ell}$ ,  $p_\ell$ )
16:   end for
17:   for each  $p_\ell \in \text{SendList}_2$  do
18:     Fill sendBuffer $_{2,\ell}$ 
19:     MPI_Isend(sendBuffer $_{2,\ell}$ ,  $p_\ell$ )
20:   end for
21:   MPI_WaitAll on all receives
22: end function

```

operations (via `MPI_Isend`) for all processes in its `SendList`. For the sake of ensuring early matching receives at each process, send buffer filling operations are performed after issuing all non-blocking receives. Once all communication operations are initiated, the process waits for their completion using `MPI_WaitAll`. This basic single-stage method is summarized in Algorithm 4.

The two-stage approach detailed in Algorithm 5 proceeds as follows. `SendList`₁ and `SendList`₂ contain the messages that will be sent in the *Communication Stage 1* and 2, respectively, as described at the end of Section 3.4. In the first stage, each process posts non-blocking receives for the incoming messages from the processes in its `RecvList`₁ and issues non-blocking sends for the processes in its `SendList`₁. The algorithm then waits for the completion of these operations, thereby ensuring that all the expand tasks that it will share are received.

If partial reduce is applied, *Computation Stage 1* follows immediately after the first communication stage. During the second communication stage, the process executes a similar sequence: it first posts receives from the processes in its `RecvList`₂, and then sends the corresponding data to the processes in its `SendList`₂.

4.3. Datasets

We conducted our experiments on two different datasets: Dataset 1 and Dataset 2. Dataset 1 is used for comparing the performance of 2spr against 1s for communication statistics. Dataset 2 is used for comparing the proposed optimization schemes in terms of both communication statistics as well as parallel runtimes.

Table 2: Average properties of the sparse matrix groups in Dataset 1.

matrix group	number of			row degree		
	matrices	rows/cols	nz	avg	max	cv
$cv \geq 1$	174	79.80K	1.21M	28.67	16.18K	7.67
$cv \geq 3$	90	86.14K	1.00M	18.74	29.67K	13.12
$cv \geq 10$	31	140.56K	1.06M	10.26	75.41K	27.97

Table 3: Properties of the sparse matrices in Dataset 2.

matrix	number of			row degree		
	rows/cols	nz	density	avg	max	cv
Reddit	233.0K	114.8M	$2.12 \cdot 10^{-3}$	493	21.7K	1.62
coAuthorsDBLP	299.1K	2.0M	$2.19 \cdot 10^{-5}$	7	336	1.50
coPapersCiteseer	434.1K	32.1M	$1.70 \cdot 10^{-4}$	74	1.2K	1.37
coPapersDBLP	540.5K	30.5M	$1.04 \cdot 10^{-4}$	56	3.3K	1.17
ogbl-ppa	576.3K	42.5M	$1.28 \cdot 10^{-4}$	74	3.2K	1.35
Yelp	716.8K	14.0M	$2.72 \cdot 10^{-5}$	19	4.9K	3.64
AmazonProducts	1.6M	264.3M	$1.07 \cdot 10^{-4}$	168	75.1K	4.98
as-Skitter	1.7M	22.2M	$7.71 \cdot 10^{-6}$	13	35.5K	10.46
ogbn-products	2.4M	123.7M	$2.06 \cdot 10^{-5}$	51	17.5K	1.90
Freescall2	3.0M	23.0M ²	$2.56 \cdot 10^{-6}$	8	30.2K	4.88
com-Orkut	3.1M	234.4M	$2.48 \cdot 10^{-5}$	76	33.3K	2.03
wikipedia-2007	3.6M	45.0M	$3.54 \cdot 10^{-6}$	13	7.1K	2.61
case15	5.2M	99.2M	$3.73 \cdot 10^{-6}$	19	47	0.30
uk-2002	18.5M	298.1M	$8.69 \cdot 10^{-7}$	16	2.5K	1.71
arabic-2005	22.7M	640.0M	$1.24 \cdot 10^{-6}$	28	9.9K	2.80

Dataset 1 consists of 174 sparse matrices obtained from the SuiteSparse Matrix Collection [10] with row/column counts ranging from 5K to 1.1M, that have row-degree coefficient of variation (cv) values greater than 1. Large cv values refer to larger irregularity in the sparsity pattern of matrices. These sparse matrices are further divided into three subgroups based on their row-degree cv values greater than 1, 3, and 10. The average properties of each subgroup are reported in Table 2. In Table 2, the number of nz refers to the number of nonzeros.

Dataset 2 contains 15 square sparse matrices with relatively large computational granularity (large number of nonzeros). Table 3 presents the properties of these matrices. The sparse matrices are displayed in increasing order with respect to their number of rows and columns. All matrices except ogbn-products, ogbl-ppa, AmazonProducts, Reddit, and Yelp are obtained from the SuiteSparse Matrix Collection [10].

The GNN datasets AmazonProducts, Reddit, and Yelp were introduced in [11], [12], and [13], respectively. ogbl-ppa and ogbn-products are obtained from Open Graph Benchmark (OGB) [14]. These GNN datasets are converted to sparse matrices, which correspond to adjacency matrix representations of their respective graphs. The number of nonzeros of these matrices is in the range from 2M to 640M.

In Table 3, the number of rows/cols and nz, respectively, refer to the number of vertices and twice the number of edges, of the respective GNN graphs. The row degree values refer to the

²Explicit zero entries are also included.

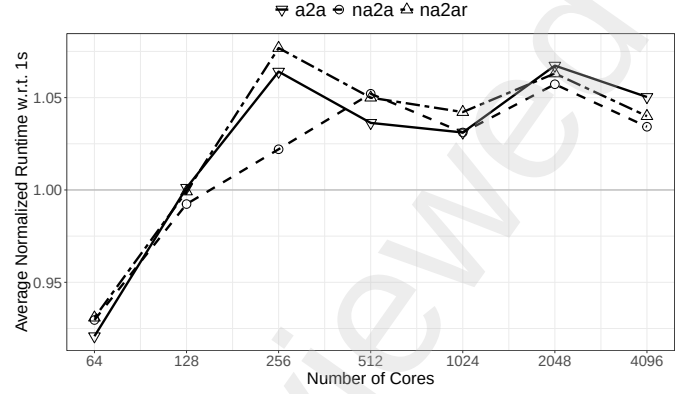


Figure 7: Average normalized runtimes of variable-size all-to-all collective routine (a2a) and neighbor collective routines (na2a, na2ar) with respect to P2P communication scheme (1s) for 15 datasets of Dataset 2.

vertex degree values.

The sparse matrices in Dataset 1 have relatively few rows/columns and nonzeros, making them unsuitable for scaling to high process counts. Therefore, for our exascale tests we used Dataset 2, which consists exclusively of large instances.

4.4. Results

Here, we provide the performance results and related discussions on the example application SpMM. In the following tables and figures, 1s refers to the conventional point-to-point (P2P) one-stage expand communication scheme, 1spr refers to the 1s scheme that utilizes the partial-reduce method, and rb1s [2] refers to 1s with a priori recursive bipartitioning for send-volume balancing. The details of rb1s are summarized in Section 5. 2s refers to the proposed two-stage expand-task sharing communication scheme, and 2spr refers to the 2s scheme that utilizes the partial-reduce method. We include 1spr and rb1s in order to show the relative performance improvement of the proposed expand-task sharing method.

4.4.1. Comparisons against MPI Collectives

We compare the conventional 1s against three other MPI-based collective alternatives on Lumi. These alternatives include the collective routine MPI_Alltoallv (a2a), the neighbor collective routine MPI_Neighbor_alltoallv (na2a), its variant with enabled reordering (na2ar). For neighbor collectives, we construct MPI-distributed graph topologies using the bipartite graphs described earlier via the MPI_Dist_graph_create_adjacent routine.

SpMM runtime results are given in Fig. 7. All runtimes are normalized with respect to 1s. Each curve represents the average normalized runtimes across Dataset 2 at varying core counts, where lower values than 1.0 indicate better performance relative to 1s.

We observe that 1s outperforms the alternative MPI-based methods at higher core counts. This performance advantage is likely due to the fact that, unlike the alternatives, which must fully prepare the send buffers before initiating communication, 1s overlaps communication and preparation by issuing

Table 4: Communication statistics as well as parallel runtimes for row-parallel SpMM instances in Dataset 2 on Lumi with 4096 cores.

matrix	algo	actual values							normalized with respect to 1s						
		Max vol load per proc			Vol.	# of mssgs.		time	Max vol load per proc			Vol.	# of mssgs.		time
		Send	Recv	S+R	avg	max	avg	(ms)	Send	Recv	S+R	avg	max	avg	(ms)
As-Skitter	1s	14.3K	35.4K	40.7K	2.9K	2.2K	252	11.0	-	-	-	-	-	-	-
	1spr	2.8K	2.8K	5.6K	1.1K	2.2K	252	7.2	0.196	0.079	0.138	0.365	1.000	1.000	0.655
	rb1s	5.1K	35.5K	36.1K	2.9K	1.8K	223	10.6	0.354	1.002	0.887	0.992	0.805	0.885	0.963
	2s	4.6K	35.4K	35.4K	2.9K	2.1K	339	9.8	0.322	1.000	0.870	1.000	0.968	1.345	0.891
	2spr	2.4K	2.8K	3.2K	1.1K	2.2K	302	5.4	0.168	0.079	0.079	0.365	0.991	1.198	0.490
cage15	1s	8.8K	10.4K	18.4K	8.9K	195	50	8.9	-	-	-	-	-	-	-
	1spr	7.7K	7.7K	15.5K	6.5K	195	50	7.0	0.875	0.743	0.842	0.722	1.000	1.000	0.787
	rb1s	7.2K	8.5K	15.2K	8.6K	282	81	8.9	0.817	0.820	0.824	0.962	1.446	1.620	1.002
	2s	7.9K	10.4K	11.0K	8.9K	311	123	7.7	0.897	1.000	0.598	1.000	1.595	2.460	0.865
	2spr	6.7K	7.7K	9.9K	6.5K	328	97	6.2	0.757	0.743	0.540	0.722	1.682	1.940	0.697
com-Orkut	1s	44.3K	49.5K	85.2K	49.4K	4.0K	2.3K	79.5	-	-	-	-	-	-	-
	1spr	33.9K	33.9K	67.9K	37.7K	4.0K	2.3K	63.0	0.765	0.685	0.797	0.763	1.000	1.000	0.793
	rb1s	45.0K	51.1K	93.8K	52.5K	3.9K	1.8K	94.7	1.016	1.033	1.100	1.062	0.972	0.800	1.191
	2s	43.6K	49.5K	49.7K	49.4K	4.5K	3.1K	64.7	0.984	1.000	0.583	1.000	1.119	1.363	0.814
	2spr	33.6K	34.0K	37.9K	37.7K	4.6K	3.1K	58.8	0.758	0.687	0.445	0.763	1.136	1.363	0.740
ogbl-ppa	1s	6.7K	7.4K	14.1K	6.8K	1.6K	643	8.1	-	-	-	-	-	-	-
	1spr	5.6K	5.6K	11.2K	4.9K	1.6K	643	7.3	0.831	0.760	0.794	0.720	1.000	1.000	0.901
	rb1s	6.6K	7.2K	12.1K	6.7K	1.8K	722	14.7	0.985	0.969	0.860	0.986	1.123	1.123	1.815
	2s	6.3K	7.4K	7.6K	6.8K	2.2K	995	7.6	0.932	1.000	0.539	1.000	1.413	1.547	0.938
	2spr	4.6K	5.6K	6.6K	4.9K	2.0K	866	6.4	0.678	0.761	0.469	0.720	1.297	1.347	0.790
ogbn-products	1s	13.7K	19.4K	24.5K	10.1K	1.6K	556	17.4	-	-	-	-	-	-	-
	1spr	9.2K	9.2K	18.4K	6.2K	1.6K	556	12.8	0.670	0.473	0.751	0.611	1.000	1.000	0.736
	rb1s	11.7K	19.7K	26.6K	9.7K	1.7K	440	14.9	0.857	1.016	1.085	0.961	1.038	0.791	0.856
	2s	9.6K	19.4K	19.5K	10.1K	2.4K	929	14.3	0.704	1.000	0.796	1.000	1.516	1.671	0.822
	2spr	5.7K	9.2K	12.5K	6.2K	2.1K	809	10.6	0.413	0.473	0.510	0.611	1.292	1.455	0.609
Reddit	1s	20.3K	23.3K	34.3K	17.6K	3.7K	1.8K	53.5	-	-	-	-	-	-	-
	1spr	13.1K	13.1K	26.2K	11.7K	3.7K	1.8K	40.6	0.645	0.562	0.764	0.665	1.000	1.000	0.759
	rb1s	16.9K	25.1K	31.4K	16.8K	3.7K	1.4K	43.8	0.833	1.078	0.916	0.954	1.001	0.789	0.819
	2s	17.3K	23.3K	23.3K	17.6K	3.9K	2.2K	41.1	0.852	1.000	0.679	1.000	1.053	1.256	0.768
	2spr	12.2K	13.1K	15.8K	11.7K	3.7K	2.0K	34.6	0.601	0.562	0.461	0.665	0.992	1.159	0.647
uk-2002	1s	26.9K	194.9K	194.9K	4.4K	2.1K	237	33.5	-	-	-	-	-	-	-
	1spr	6.6K	11.2K	15.7K	1.0K	2.0K	237	12.9	0.246	0.057	0.081	0.227	1.000	1.000	0.385
	rb1s	51.7K	193.7K	204.0K	13.6K	2.4K	146	41.1	1.921	0.994	1.047	3.087	1.145	0.616	1.227
	2s	24.3K	194.9K	194.9K	4.4K	1.4K	273	32.5	0.903	1.000	1.000	1.000	0.667	1.152	0.970
	2spr	3.7K	11.2K	13.1K	1.0K	1.6K	247	11.1	0.138	0.057	0.067	0.227	0.762	1.042	0.331

1s: conventional one-stage, 1spr: 1s scheme with partial reduce, rb1s: RB-based 1s [2], 2s: the proposed 2-stage expand-task sharing, and 2spr: the proposed 2s scheme utilizing partial reduce. Communication volumes are displayed in terms of X-/Y-matrix rows.

MPI_Isend operations on the fly (see Algorithm 4). Given the superior performance of 1s at higher core counts, we restrict our comparisons for the proposed methods relative to 1s.

4.4.2. Performance of Expand-Task Sharing

We provide Table 4 to show the corroboration between the improvements obtained in communication metrics provided in Section 3.1 and the improvements obtained in parallel runtimes. In this setting, Table 4 shows communication statistics as well as parallel runtimes for row-parallel SpMM instances with $f = 64$ using different communication schemes on Lumi with 4096 processes. The communication volume values are displayed in terms of X-/Y-matrix rows, whereas parallel runtimes are given in terms of milliseconds. The communication volumes in bytes can be obtained by multiplying the displayed values by 64×8 since the width of the dense matrices f is 64 and double-

precision floating-point words are used.

In Table 4, the columns 3–9 show the actual values, whereas the columns 10–16 show the values normalized with respect to those of 1s. The bottom three rows of the table show the geometric averages of the actual values as well as the normalized values over the seven SpMM instances.

We first compare our proposed expand-task sharing scheme 2s against the most notable communication volume balancing scheme [2] named rb1s. The trade-off between 2s and rb1s is as follows. Both methods suffer from a higher number of messages. rb1s suffers from higher communication volume, possibly due to better performance of K-way partitioning, and mainly the additional constraint introduced by the scheme, reducing the solution space. 2s scheme can be applied after any partitioning phase and directly balances communication volume without increasing the total volume.

Table 5: Communication statistics for different algorithms and core counts on Dataset 1, grouped by row cv thresholds.

group	cores	algo	actual values							normalized with respect to 1s						
			# of mssgs.		max vol load per proc			vol (S+R)		# of mssgs.		max vol load per proc			vol (S+R)	
			max	avg	send	recv	S+R	cv	avg	max	avg	send	recv	S+R	cv	avg
cv ≥ 1	64	1s	59.00	31.84	3.74K	15.56K	17.26K	0.78	4.47K	-	-	-	-	-	-	-
		2spr	72.03	41.44	1.12K	1.23K	2.03K	0.31	1.57K	1.22	1.30	0.30	0.08	0.12	0.39	0.35
	128	1s	112.95	47.34	2.75K	15.08K	16.13K	1.01	2.78K	-	-	-	-	-	-	-
		2spr	123.46	57.83	0.86K	0.94K	1.48K	0.35	1.08K	1.09	1.22	0.31	0.06	0.09	0.35	0.39
	256	1s	206.80	62.68	2.23K	14.75K	15.44K	1.31	1.81K	-	-	-	-	-	-	-
		2spr	207.03	72.17	0.67K	0.79K	1.14K	0.45	0.75K	1.00	1.15	0.30	0.05	0.07	0.34	0.41
	512	1s	350.45	74.52	1.99K	14.50K	15.09K	1.70	1.11K	-	-	-	-	-	-	-
		2spr	344.72	81.24	0.63K	0.76K	1.00K	0.63	0.51K	0.98	1.09	0.32	0.05	0.07	0.37	0.46
	1024	1s	568.98	79.33	2.04K	14.38K	15.07K	2.25	0.67K	-	-	-	-	-	-	-
		2spr	559.56	83.21	0.72K	0.83K	1.02K	0.93	0.35K	0.98	1.05	0.35	0.06	0.07	0.41	0.52
cv ≥ 3	64	1s	59.72	25.96	4.69K	26.54K	27.77K	1.19	5.28K	-	-	-	-	-	-	-
		2spr	66.56	31.79	0.76K	0.84K	1.38K	0.44	0.99K	1.11	1.22	0.16	0.03	0.05	0.37	0.19
	128	1s	115.68	35.96	3.36K	26.26K	26.81K	1.60	3.02K	-	-	-	-	-	-	-
		2spr	120.53	42.84	0.60K	0.68K	1.04K	0.51	0.70K	1.04	1.19	0.18	0.03	0.04	0.32	0.23
	256	1s	219.93	47.10	2.83K	26.03K	26.30K	2.09	1.87K	-	-	-	-	-	-	-
		2spr	217.97	53.82	0.55K	0.67K	0.90K	0.68	0.48K	0.99	1.14	0.19	0.03	0.03	0.32	0.26
	512	1s	403.64	54.17	2.66K	25.95K	26.33K	2.72	1.05K	-	-	-	-	-	-	-
		2spr	391.44	58.77	0.57K	0.76K	0.95K	0.94	0.32K	0.97	1.08	0.22	0.03	0.04	0.34	0.31
	1024	1s	714.84	56.18	2.94K	25.95K	26.68K	3.56	0.58K	-	-	-	-	-	-	-
		2spr	702.20	58.98	0.79K	0.96K	1.14K	1.36	0.22K	0.98	1.05	0.27	0.04	0.04	0.38	0.37
cv ≥ 10	64	1s	59.42	23.03	7.10K	63.57K	63.69K	1.74	6.62K	-	-	-	-	-	-	-
		2spr	69.94	28.45	0.55K	0.70K	1.16K	0.77	0.51K	1.18	1.24	0.08	0.01	0.02	0.44	0.08
	128	1s	115.68	30.16	5.50K	63.57K	63.68K	2.48	3.45K	-	-	-	-	-	-	-
		2spr	125.65	36.48	0.43K	0.61K	0.96K	0.93	0.35K	1.09	1.21	0.08	0.01	0.02	0.37	0.10
	256	1s	223.03	34.97	4.83K	63.57K	63.79K	3.45	1.77K	-	-	-	-	-	-	-
		2spr	220.55	39.29	0.37K	0.61K	0.84K	1.22	0.21K	0.99	1.12	0.08	0.01	0.01	0.35	0.12
	512	1s	432.87	33.26	4.44K	63.57K	64.00K	4.73	0.91K	-	-	-	-	-	-	-
		2spr	425.32	35.71	0.54K	0.80K	1.01K	1.70	0.14K	0.98	1.07	0.12	0.01	0.02	0.36	0.15
	1024	1s	843.00	27.48	4.61K	63.57K	64.41K	6.25	0.48K	-	-	-	-	-	-	-
		2spr	833.06	29.61	0.88K	1.00K	1.21K	2.29	94.94	0.99	1.08	0.19	0.02	0.02	0.37	0.20

Communication volumes are displayed in terms of X-/Y-matrix rows where values in bytes can be obtained by multiplying them with 512.

As seen in Table 4, improvement rates of 2s over 1s communication schemes in $\max(\text{SendVol} + \text{RecvVol})$ volume metric (Eq. 10) strongly relate to the improvement rates over the parallel runtimes. For example, on average, 29% improvement attained by 2s over 1s in $\max(\text{SendVol} + \text{RecvVol})$ metric without increasing the total volume reflects as 14% improvement on parallel runtime. A similar corroboration exists between 2spr and 1spr schemes in terms of their maximum communication volume metrics and parallel runtimes. For example, on average, 36% improvement attained by 2spr over 1spr in $\max(\text{SendVol} + \text{RecvVol})$ volume metric reflects as 16% improvement on parallel runtime.

When the expand-task sharing method is applied after the partial-reduce method, the receive-volume bottleneck problem disappears. Therefore, expand-task sharing achieves a higher rate of improvement in $\max(\text{SendVol} + \text{RecvVol})$ metric (36% improvement of 2spr over 1spr versus 29% improvement of 2s over 1s, on average).

2spr attains significant improvement of 72% in $\max(\text{SendVol} + \text{RecvVol})$ as well as 46% improvement in total volume over 1s. These significant improvements of 2spr

over 1s in communication cost metrics lead to 41% improvement on parallel runtime, on average.

The results on the non-symmetric matrix uk-2002 deserve a special discussion. For this SpMM instance, the maximum volume load of 1s is dominated by the receive volume load ($\max(\text{SendVol}) = 27\text{K}$ versus $\max(\text{RecvVol}) = 195\text{K}$). Although 2s decreases the maximum send volume metric by 10% over 1s, it fails to decrease the $\max(\text{SendVol} + \text{RecvVol})$ metric as it is dominated by the maximum receive volume. So, this instance justifies our motivation behind introducing the partial-reduce method since the vertex-cover model successfully decreases the maximum receive volume from 195K to 11K. Thus, 2spr achieves a 67% runtime improvement.

4.4.3. Impact of Optimization Methods on Scale-Free Matrices

Table 5 presents the communication performance of 2spr compared to 1s for matrix groups in Dataset 1. The table reports the average results across the matrices in each group.

The left side of the table displays the actual values, whereas the right-hand side displays the normalized values with respect to 1s. Here, the proposed 2spr significantly decreases the com-

munication volume of the bottleneck process. This performance gap in terms of communication volume balancing increases with increasing cv for fixed process count in favor of 2spr. These experimental findings on communication balancing performance variation reflect on parallel-SpMM running time performance on Lumi. For example, on 256 cores and $f = 64$, 2spr runs 2.3 $\times$, 3.0 $\times$, and 4.5 $\times$ faster over 1s for the $cv \geq 1$, $cv \geq 3$, and $cv \geq 10$ matrix groups, respectively. As noted earlier, since Dataset 1 is composed of relatively small matrices, the scalability of both 1s and 2spr saturates and does not improve beyond 256 processes.

4.4.4. Strong Scaling Experiments

Figure 8 shows the results of the strong scaling experiments as the variation of parallel runtimes with increasing number of processes/cores. Figure 8a displays the strong scaling curves for the 15 SpMM instances of Dataset 2 given in Table 3 on Lumi with the number of cores ranging from 64 to 4096. Figure 8b displays the strong scaling curves for 10 SpMM instances out of 15 on MareNostrum 5 with the number of cores ranging from 56 to 7168. On Lumi and MareNostrum 5, we respectively used $f = 64$ and $f = 512$.

As seen in Fig. 8a, on Lumi, the proposed 2spr scheme performs much better scaling than the conventional 1s scheme. 2spr continues to scale until 4096 cores on 14 out of 15 SpMM instances, whereas on the other SpMM instance FreeScale2, it continues to scale until 2048 cores.

As seen in Fig. 8b, on MareNostrum 5, 2spr performs much better scaling than 1s. 2spr continues to scale until 7168 cores on 4 out of 10 SpMM instances, whereas on the other SpMM instances, it continues to scale until 3584 cores. Note that 2spr successfully scales FreeScale2 and Yelp on Lumi, as well as as-Skitter and FreeScale2 on MareNostrum 5, where 1s fails to do so.

5. Related Work

Total communication volume is the most widely optimized communication cost metric for improving the performance of irregularly sparse applications on distributed-memory systems. Total volume can already be effectively minimized with existing partitioners [15, 1]. However, scaling irregular applications on distributed-memory systems necessitates correct encapsulation and optimization of multiple communication cost metrics in those applications. These cost metrics can be categorized under two groups: metrics related to bandwidth costs (such as total and maximum communication volume) and latency costs (such as total and maximum message count). A few earlier works [16, 17] consider communication cost metrics other than total volume in a two-phase framework, where the partitioning in the first phase that minimizes total communication volume is followed by a latter phase in which they address minimizing other metrics. However, these works suffer from not obtaining conformal input output partitions since they all utilize the flexibility in non-conformal input output partitions to address other communication costs. Conformal input/output partitioning is

important in the parallelization of sparse iterative methods to avoid extra communication between successive iterations. We have recently proposed and implemented one-phase approaches [18, 19, 2, 20] that encapsulate minimizing multiple communication cost metrics while maintaining conformal input/output partitioning.

This work addresses bandwidth cost in parallelizing sparse irregular operations such as SpMM, SpGEMM, sparse MT-TKRP, and GNN training. Although these applications incur sparse P2P communications, the amount of data communicated is sufficiently large, so latency-based communication costs can be negligible compared to bandwidth-related costs. Since interconnection topologies in modern tier-0 HPC systems have very low diameters, they do not suffer from message congestion, thus avoiding the need for topology-aware mapping [21]. Therefore, assuming a homogeneous HPC system, scaling a parallel application aims to reduce the computation and communication loads of the most heavily loaded processors. Since total computational cost is usually constant, the former metric reduces to computational load balancing. However, total communication volume depends on partitioning, so communication volume balancing should be modeled together with total volume minimization. In [2], we proposed and implemented a comprehensive and generic one-phase framework, enabling conformal input-output partitioning to minimize multiple volume-based communication cost metrics. This framework relies on the widely adopted recursive bipartitioning (RB) paradigm utilized in graph/hypergraph partitioning. Computational load balancing and total volume minimization are achieved with existing partitioners. During the RB process, communication volume load balancing is achieved via a two-constraint formulation, which assigns communication volume weight to boundary vertices to use the readily available partitioner.

Kayaaslan, Ucar, and Aykanat proposed [22, 23] semi-2D parallel SpMV method that involves heterogeneous messages to prevent message fragmentation problems introduced by conventional 2D fine-grain sparse matrix partitioning through a one-phase communication stage. In this scheme, the authors utilize 1D partitioning together with the vertex cover model to further decrease the communication volume of the 1D partition. 1spr and 2spr schemes proposed in the current work, utilize and enhance the vertex-cover model in [22, 23] for decreasing the receive volume loads of processes, where the enhancement refers to maximizing the flexibility of expand-task sharing and minimizing the computation imbalance introduced by partial reduce.

A few works [24, 25] explore sparsity-agnostic algorithms, whose concepts are inspired by dense GEMM methods such as SUMMA [26] and Cannon [27]. These algorithms operate by broadcasting, shifting, and reducing block rows and block columns of the sparse and dense matrices. We tested the 1.5D sparse-shift dense-replicate algorithm for distributed SpMM introduced by Bharadwaj et al. [25]. Our experiments on MareNostrum 5 showed a significant performance advantage for our method, which is expected: when the sparsity pattern is not exploited, the replicated dense submatrices are underutilized compared to our smart point-to-point communication ap-

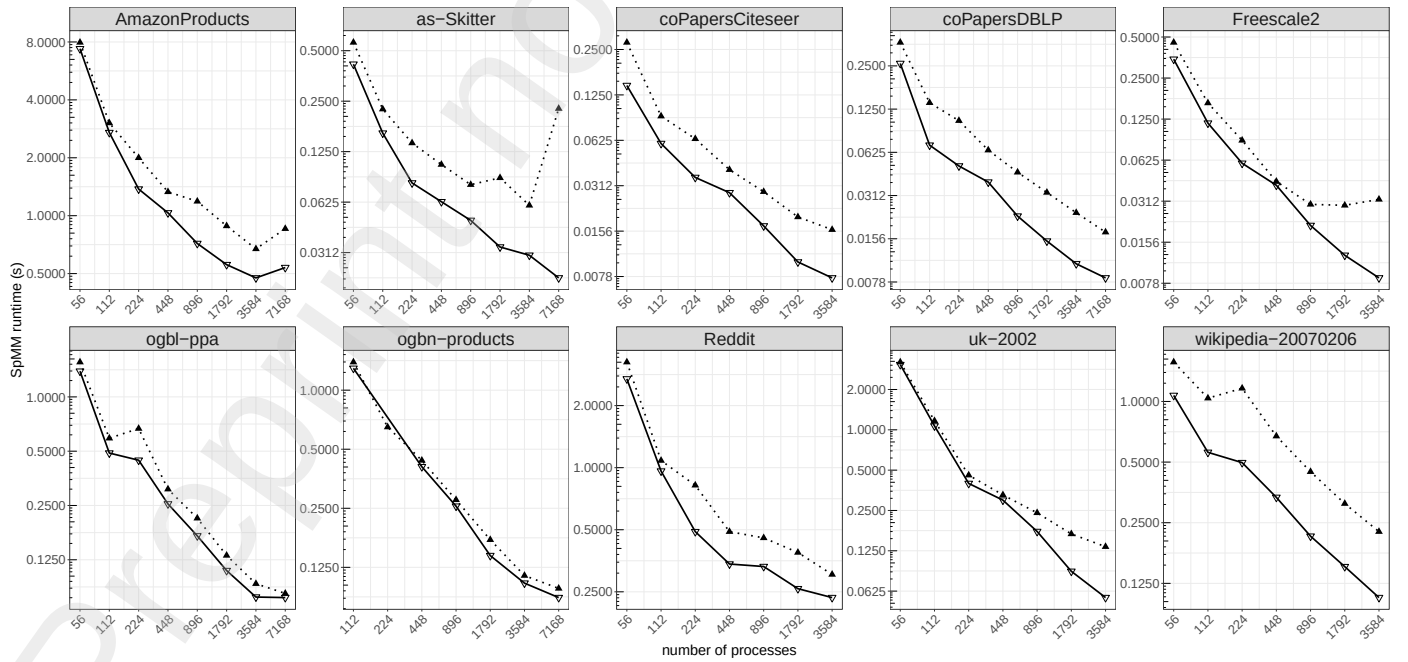
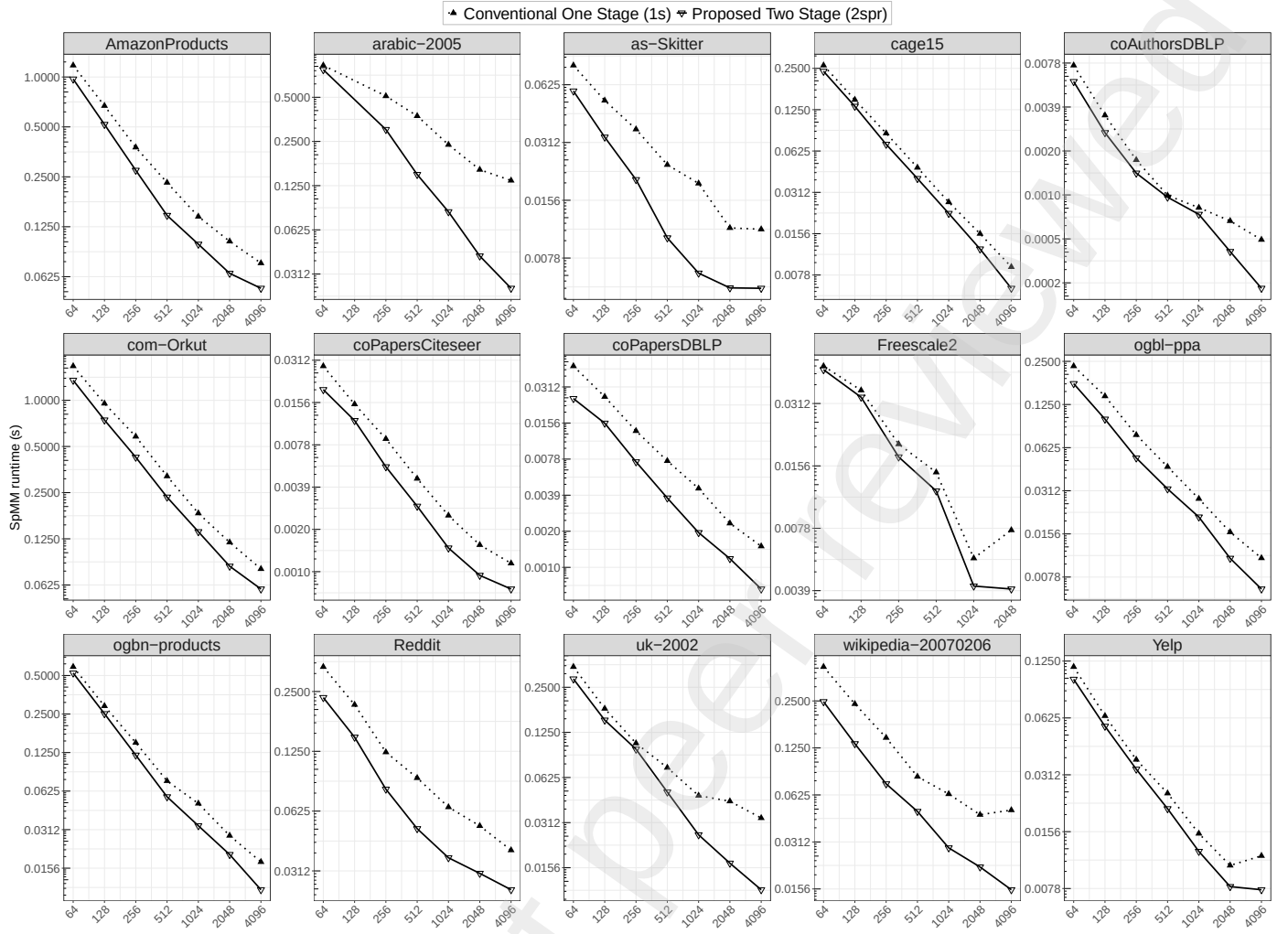


Figure 8: SpMM strong scaling curves of Dataset 2 on Lumi and MareNostrum 5.

proach that groups common neighbors within the same process. In this sense, the expand operation can also be seen as a form of sparsity-aware smart replication where it selectively replicates the input dense submatrices. The advantage of these methods is that they do not require a sophisticated graph partitioning step.

6. Conclusion

We investigated the efficient parallelization of irregularly sparse bandwidth-bound applications that involve sparse collectives on distributed-memory HPC systems. We proposed two communication optimization methods for decreasing the bandwidth overhead of such collectives by minimizing the volume load of bottleneck processes. For the first optimization, we proposed a novel expand-task sharing method for balancing the communication volume loads of processes by changing their send volume loads. For the second optimization, we introduced a vertex-cover-based partial reduce method for decreasing the processes' receive volume loads. Extensive experiments performed through the parallelization of the SpMM application on two large-scale HPC systems led to the following findings:

- The first optimization method already achieves significant improvements in send-volume bound SpMM instances.
- The second optimization method further attains significant improvements over the first one for receive-volume bound SpMM instances.
- The two optimization methods adopted together increase the bandwidth-bound applications' scalability. For example, the combined method decreases the parallel runtime of SpMM instance `arabic-2005` on 4096 cores by a factor of 5.

As future work, we plan to investigate incorporating node awareness into the expand-task sharing optimization method and adapting our methods to other parallelization strategies, namely column-parallel and 2D SpMM. We also aim to explore partial-reduce methods that match different partial reduces on unweighted (boolean) adjacency matrices.

Acknowledgements

This work is supported by the Inno4scale project CVolBal, which is funded by the European High-Performance Computing Joint Undertaking (JU) under Grant Agreement No 10111813. The JU receives support from the European Union's Horizon Europe Programme.

Pre-exascale supercomputing systems Lumi and MareNostrum 5 were used for performance evaluations. We thank the Lumi project EHPC-DEV-2024D05-003 for providing access to Lumi. We also thank the Barcelona Supercomputing Center (BSC). Lumi resources have been used under the Lumi project EHPC-DEV-2024D05-003. We thank the Scientific and Technological Research Council of Turkey (TÜBİTAK) for providing access to MareNostrum 5. We also thank TÜBİTAK for using their Truba Arf system for parallel code development and testing.

Appendix A.

This appendix provides the proof for the correctness of Algorithm 3. Given a bipartite graph $G = (V = A \cup B, E)$ and a maximum cardinality matching (MCM) M , the DM decomposition is a partition of the vertices V into three disjoint and unique sets $V = \{O, N, R\}$ such that O and N are the set of vertices that are reachable from unmatched vertices by M through alternating paths of odd and even lengths, respectively. R is the set of remaining vertices [28]. We present some important properties of these sets in the following three lemmas, without proof; the reader is referred to [29] for their proofs.

Lemma 1. *There is no edge between N and $N \cup R$.*

Lemma 2. *O and R are always matched by M*

Lemma 3. *The matched edges join either two vertices in R or a vertex in O with a vertex in N .*

We provide the following corollary.

Corollary 1. *Both $O \cup R_A$ and $O \cup R_B$ form an MVC, where $R_A = R \cap A$ and $R_B = R \cap B$.*

Proof: By Lemma 1, $O \cup R_A$ and $O \cup R_B$ cover all edges. The vertex covers they form are also minimum since their cardinality is equal to $|M|$ [30]. Note that R introduces flexibility in choosing an MVC suitable for different objectives [31]. $\square$

Lemma 4. *O is in every MVC of G .*

Proof: In an MVC, exactly one of the endpoints of edges in M must be selected [32]. Each unmatched vertex $n \in N$ must have at least one edge with a vertex $o \in O$ (if there is no such vertex, $O = N = \emptyset$). For each such edge, o must be in every MVC as n is unmatched and o is matched (Lemma 2). Consequently, none of the matched vertices in N can be in an MVC as both ends of a matched edge would be chosen (Lemma 3). Therefore, O is always in an MVC. $\square$

Finally, we prove the main theorem.

Theorem 1. *$C = R_A \cup O$ is the MVC with the maximum cardinality of C_A among all the MVCs of the bipartite graph G .*

Proof: Since O is always in an MVC (Lemma 4), choosing the remaining vertices from R_A forms the MVC with the maximum number of vertices from A . $\square$

Algorithm 3 finds the C stated in Theorem 1 as it finds the vertices reachable from alternating paths starting from unmatched vertices in A . Thus,

$$U = (O \cap B) \cup (N \cap A) = O_B \cup N_A. \quad (\text{A.1})$$

C computed and returned by Algorithm 3 is then equal to $R_A \cup O$. Conversely, alternating paths starting from unmatched vertices in B would find U and C such that $U = O_A \cup N_B$, $C = R_B \cup O$.

References

- [1] G. Karypis, V. Kumar, Metis—a software package for partitioning unstructured graphs, partitioning meshes and computing fill-reducing ordering of sparse matrices (01 1997).
- [2] S. Acer, O. Selvitopi, C. Aykanat, Improving performance of sparse matrix dense matrix multiplication on large-scale parallel systems, *Parallel Computing* 59 (2016) 71–96, theory and Practice of Irregular Applications. doi:<https://doi.org/10.1016/j.parco.2016.10.001>.
- [3] K. Akbudak, O. Selvitopi, C. Aykanat, Partitioning models for scaling parallel sparse matrix-matrix multiplication, *ACM Trans. Parallel Comput.* 4 (3) (Jan. 2018). doi:10.1145/3155292. URL <https://doi.org/10.1145/3155292>
- [4] K. Akbudak, C. Aykanat, Simultaneous input and output matrix partitioning for outer-product-parallel sparse matrix-matrix multiplication, *SIAM Journal on Scientific Computing* 36 (5) (2014) C568–C590. arXiv:<https://doi.org/10.1137/13092589X>, doi:10.1137/13092589X. URL <https://doi.org/10.1137/13092589X>
- [5] B. Uçar, C. Aykanat, Partitioning sparse matrices for parallel preconditioned iterative methods, *SIAM Journal on Scientific Computing* 29 (4) (2007) 1683–1709. arXiv:<https://doi.org/10.1137/040617431>, doi:10.1137/040617431. URL <https://doi.org/10.1137/040617431>
- [6] J. E. Hopcroft, R. M. Karp, An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs, *SIAM Journal on Computing* 2 (4) (1973) 225–231. doi:10.1137/0202019.
- [7] P. Sanders, C. Schulz, Think Locally, Act Globally: Highly Balanced Graph Partitioning, in: *Proceedings of the 12th International Symposium on Experimental Algorithms (SEA'13)*, Vol. 7933 of LNCS, Springer, 2013, pp. 164–175.
- [8] H. Meyerhenke, P. Sanders, C. Schulz, *Parallel Graph Partitioning for Complex Networks*, Vol. 28, 2017, pp. 2625–2638.
- [9] T. Hoefler, A. Lumsdaine, Optimizing non-blocking collective operations for infiniband, in: *2008 IEEE International Symposium on Parallel and Distributed Processing*, IEEE, 2008, pp. 1–8.
- [10] T. A. Davis, Y. Hu, The University of Florida Sparse Matrix Collection, *ACM Trans. Math. Softw.* 38 (1) (2011) 1:1–1:25. doi:10.1145/2049662.2049663.
- [11] W. L. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems*, Vol. 30, 2017, pp. 1025–1035.
- [12] H. Zeng, J. Zhou, A. Srivastava, R. Kannan, V. Prasanna, Graphsaint: Graph sampling based inductive learning method, arXiv preprint arXiv:1907.04931 (2019).
- [13] J. McAuley, J. Leskovec, Hidden factors and hidden topics: Understanding rating dimensions with review text, in: *Proceedings of the 7th ACM Conference on Recommender Systems*, 2013, pp. 165–172.
- [14] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec, Open graph benchmark: Datasets for machine learning on graphs, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 22118–22133.
- [15] U. Catalyurek, C. Aykanat, Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication, *IEEE Transactions on Parallel and Distributed Systems* 10 (7) (1999) 673–693. doi:10.1109/71.780863.
- [16] B. Uçar, C. Aykanat, Encapsulating multiple communication-cost metrics in partitioning sparse rectangular matrices for parallel matrix-vector multiplies, *SIAM Journal on Scientific Computing* 25 (6) (2004) 1837–1859. doi:10.1137/S1064827502410463.
- [17] R. H. Bisseling, W. Meesen, Communication balancing in parallel sparse matrix-vector multiplication, *Electronic Transactions on Numerical Analysis* 21 (2005) 47–65.
- [18] O. Selvitopi, S. Acer, C. Aykanat, A recursive hypergraph bipartitioning framework for reducing bandwidth and latency costs simultaneously, *IEEE Transactions on Parallel and Distributed Systems* 28 (2) (2017) 345–358. doi:10.1109/TPDS.2016.2577024.
- [19] S. Acer, O. Selvitopi, C. Aykanat, Optimizing nonzero-based sparse matrix partitioning models via reducing latency, *Journal of Parallel and Distributed Computing* 122 (2018) 145–158. doi:<https://doi.org/10.1016/j.jpdc.2018.08.005>.
- [20] C. Aykanat, Models for optimizing multiple communication cost metrics in parallelizing irregular applications, in: *SIAM Workshop on Combinatorial Scientific Computing*, Invited talk: https://esc20.unijena.de/speakers_aykanat.html, 11–13 Feb, 2020.
- [21] F. Pellegrini, J. Roman, Scotch: A software package for static mapping by dual recursive bipartitioning of process and architecture graphs, in: *High-Performance Computing and Networking: International Conference and Exhibition HPCN EUROPE 1996 Brussels, Belgium, April 15–19, 1996 Proceedings* 4, Springer, 1996, pp. 493–498.
- [22] E. Kayaaslan, C. Aykanat, B. Uçar, 1.5d parallel sparse matrix-vector multiply, *SIAM Journal on Scientific Computing* 40 (1) (2018) C25–C46. arXiv:<https://doi.org/10.1137/16M1105591>, doi:10.1137/16M1105591.
- [23] E. Kayaaslan, B. Uçar, C. Aykanat, Semi-two-dimensional partitioning for parallel sparse matrix-vector multiplication, in: *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*, 2015, pp. 1125–1134. doi:10.1109/IPDPSW.2015.20.
- [24] P. Koanantakool, A. Azad, A. Buluç, D. Morozov, S.-Y. Oh, L. Oliker, K. Yelick, Communication-avoiding parallel sparse-dense matrix-matrix multiplication, in: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 842–853. doi:10.1109/IPDPS.2016.117.
- [25] V. Bharadwaj, A. Buluç, J. Demmel, Distributed-memory sparse kernels for machine learning, in: *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2022, pp. 47–58. doi:10.1109/IPDPS53621.2022.00014.
- [26] R. A. van de Geijn, J. Watts, Summa: Scalable universal matrix multiplication algorithm, Technical report, University of Texas at Austin, USA (1995).
- [27] L. E. Cannon, A cellular computer to implement the kalman filter algorithm, Ph.D. thesis, University of Illinois at Urbana-Champaign, USA, ph.D. dissertation, aAI7010025 (1969).
- [28] A. L. Dulmage, N. S. Mendelsohn, Coverings of bipartite graphs, *Canadian Journal of Mathematics* 10 (1958) 517–534. doi:10.4153/CJM-1958-052-0.
- [29] A. Pothén, Sparse null bases and marriage theorems, Ph.D. thesis, Cornell University, Department of Computer Science, Ithaca, New York (1984).
- [30] R. W. Irving, T. Kavitha, K. Mehlhorn, D. Michail, K. E. Paluch, Rank-maximal matchings, *ACM Trans. Algorithms* 2 (4) (2006) 602–610. doi:10.1145/1198513.1198520.
- [31] A. Pothén, C.-J. Fan, Computing the block triangular form of a sparse matrix, *ACM Trans. Math. Softw.* 16 (4) (1990) 303–324. doi:10.1145/98267.98287.
- [32] G. Szárnyas, *Graphs and matrices: A translation of "graphok és matrixok" by dénes König (1931) (2020)*. arXiv:2009.03780.